SQL 파이썬 연동

01

Python과 MySQL 연동

mysql.connector

- Python에서 MySQL 데이터베이스와 상호작용하기 위한 공식 MySQL Connector 라이브러리
- Oracle에서 제공하는 공식 드라이버이며, Python 코드에서 MySQL 서버에 접속하고, 데이터를 조회하거나 변경하는 등의 작업을 수행할 수 있음
- 이 라이브러리를 사용하면 Python 코드 내에서 직접 데이터베이스 작업을 수행할 수 있어, 데이터 분석, 웹 개발 등 다양한 애플리케이션에서 활용
- mysql.connector 를 사용하기 위해서는 Python 환경에 라이브러리를 설치해야함
  - 설치: uv add mysql-connector-python

데이터베이스 연결

- MySQL 데이터베이스에 연결하려면 **데이터베이스 서버 주소, 사용자 이름, 비밀번호, 그리고 접속할 데이터베이스 이름**이 필요
- 일반적으로 SQL은 데이터를 조회하고 조작하는 역할을 하며, 데이터 분석이나 시각화는 수행하지 않음
- 따라서 데이터를 가져온 후, 분석 및 시각화를 위해 Python과 같은 별도의 도구를 활용

```
import mysql.connector
conn = mysql.connector.connect(
host='localhost', # 데이터베이스 서버 주소
user='root', # 데이터베이스 사용자 이름
password='비밀번호', # 데이터베이스 비밀번호
database='데이터베이스_이름' # 접속하고자 하는 데이터베이스
)
```

커서 생성

- 데이터베이스 연결을 통해 커서 객체 생성
- 커서는 SQL 쿼리를 실행하고, 결과를 가져오는 데 사용
- 커서 객체는 다양한 메서드를 제공하여 데이터베이스 작업을 수행할 수 있게 해줌
- 커서를 생성해야만 쿼리를 실행할 수 있고, 그 쿼리로 생긴 결과를 또 어떻게 가져오는지 확인 가능

```
conn.cursor()
```

쿼리 실행 및 결과 가져오기

- 쿼리 실행
  - cursor.execute(sql) : sql은 실행할 SQL 쿼리 문자열
  - cursor.description : 마지막으로 실행된 SELECT 문의 결과 컬럼에 대한 정보를 담고 있는 튜플

- 쿼리 결과 가져오기
  - cursor.fetchone(): 쿼리 결과에서 다음 행을 가져옴. 결과가 더 이상 없을 경우 None을 반환.
  - cursor.fetchall(): 쿼리 결과의 모든 행을 가져옴. 결과 집합이 큰 경우 메모리 문제를 일으킬 수 있으므로 주의가 필요.
  - cursor.fetchmany(size=숫자): 쿼리 결과에서 지정된 수(size)의 행을 가져옴. size를 지정하지 않으면 cursor.arraysize의 값이 기본적으로 사용.

연결 및 커서 종료

- 연결 및 커서 종료
  - cursor.close(): 커서 객체를 닫고, 커서에 할당된 모든 리소스를 해제. 커서를 닫은 후에는 더 이상 해당 커서를 사용할 수 없음.
  - conn.close(): 데이터베이스 연결을 종료하고, 연결에 할당된 모든 리소스를 해제. 연결을 닫은 후에는 더 이상 해당 연결을 통해 데이터베이스 작업을 수행할 수 없음.

Dataframe에 저장

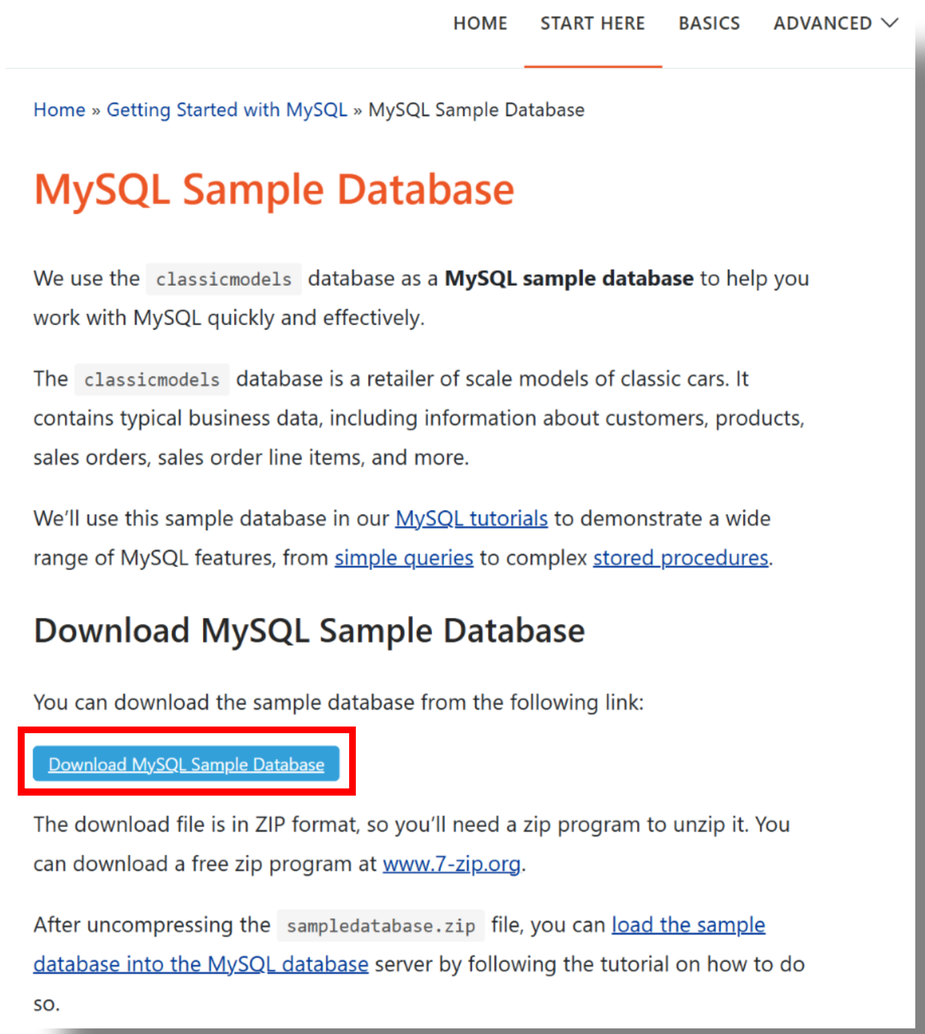
- pd.read_sql_query 함수를 사용하여 SQL 쿼리를 실행하고 결과를 DataFrame으로 가져올 수 있음
- 이렇게 하면 query에 지정된 SQL 쿼리를 실행한 결과를 df라는 DataFrame 객체에 저장할 수 있음
- 또는 fecth의 결과를 pd.dataFrame()으로 dataframe 형식으로 변경할 수 있음

```
query = "SELECT * FROM your_table"
df = pd.read_sql_query(query, conn)
```

02

Mysql tutorial

MySQL SAMPLE Database



HOME START HERE BASICS ADVANCED

Home » Getting Started with MySQL » MySQL Sample Database

MySQL Sample Database

We use the `classicmodels` database as a **MySQL sample database** to help you work with MySQL quickly and effectively.

The `classicmodels` database is a retailer of scale models of classic cars. It contains typical business data, including information about customers, products, sales orders, sales order line items, and more.

We'll use this sample database in our MySQL tutorials to demonstrate a wide range of MySQL features, from simple queries to complex stored procedures.

Download MySQL Sample Database

You can download the sample database from the following link:

Download MySQL Sample Database

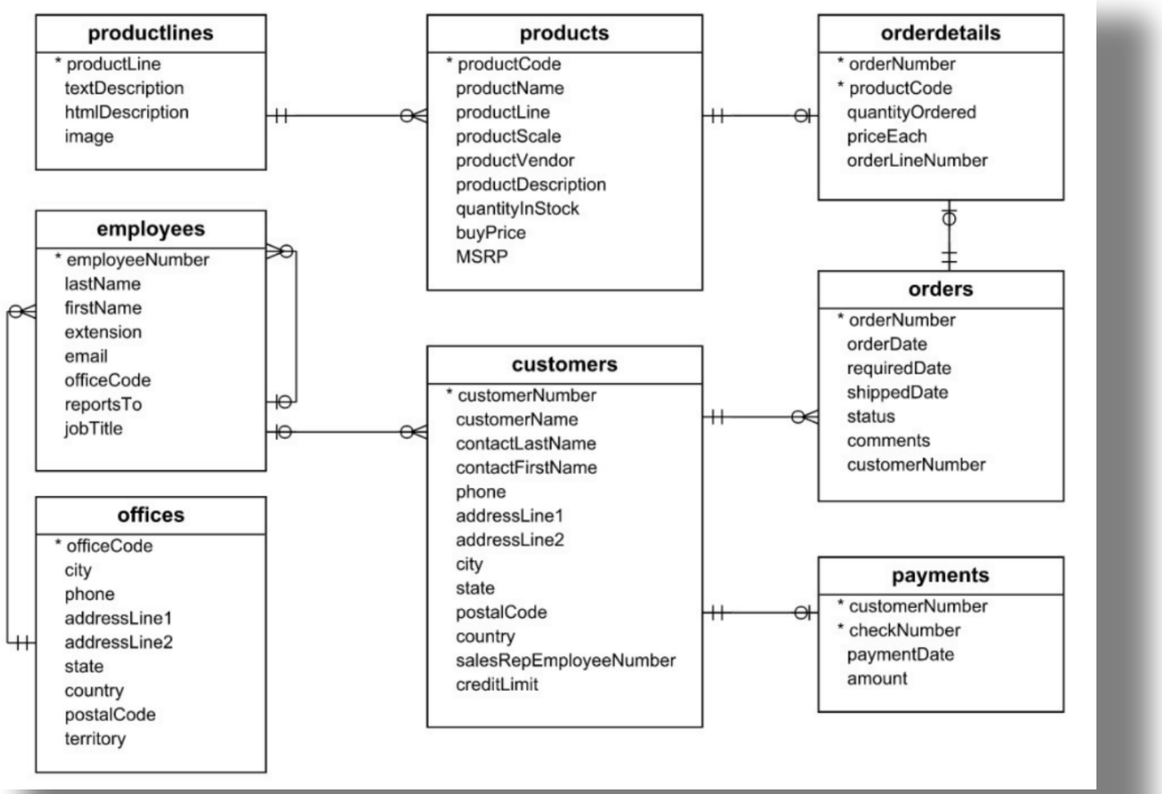
The download file is in ZIP format, so you'll need a zip program to unzip it. You can download a free zip program at www.7-zip.org.

After uncompressing the `sampledatabase.zip` file, you can load the sample database into the MySQL database server by following the tutorial on how to do so.

- MySQL SAMPLE Database(샘플 데이터베이스)는 MySQL에서 데이터베이스 학습 및 테스트를 위해 제공하는 데이터베이스
- 이를 활용하면 데이터베이스 설계, SQL 쿼리 작성, 데이터 분석 및 활용 연습을 쉽게 할 수 있음

- https://www.mysqltutorial.org/getting-started-with-mysql/mysql-sample-database/

MySQL SAMPLE Database



- MySQLTUTORIAL에서 제공하는 sample database의 구성

MySQL SAMPLE Database

- customers: 고객 정보를 담고 있으며, 고객 번호, 고객 이름, 연락처 정보 등을 포함
- orders: 주문 정보를 담고 있으며, 주문 번호, 주문 날짜, 배송 날짜, 주문 상태 등의 정보를 포함
- orderdetails: 각 주문의 상세 정보를 담고 있으며, 제품 코드, 주문된 수량, 가격 등을 포함
- products: 판매되는 제품 정보를 담고 있으며, 제품 코드, 제품 이름, 제품 라인, 재고 수량, 구매 가격, MSRP(권장 소매 가격) 등을 포함
- productlines: 제품 라인(카테고리) 정보를 담고 있으며, 제품 라인 설명, 제품 라인에 속한 제품들의 이미지 등을 포함
- employees: 회사 직원 정보를 담고 있으며, 직원 번호, 직원의 성과 이름, 직위, 상사, 근무 사무실 등을 포함
- offices: 회사 사무실 정보를 담고 있으며, 사무실 코드, 사무실 위치, 전화번호, 국가 등을 포함
- payments: 고객의 지불 정보를 담고 있으며, 지불 방식, 지불 날짜, 금액 등을 포함